

Path-Gradient Estimators for Continuous Normalizing Flows

Lorenz Vaitl, Kim Nicoli, Shinichi Nakajima, Pan Kessel

Machine Learning Group, Department of Electrical Engineering & Computer Science, Technische Universität Berlin, Germany
BIFOLD - Berlin Institute for the Foundations of Learning and Data, Technische Universität Berlin, Berlin, Germany
RIKEN Center for AIP, 103-0027 Tokyo, Chuo City, Japan

May 24, 2026

Navigation icons

Lorenz Vaitl et al. (TU Berlin)

Path-Gradient Estimators for CNFs...

May 24, 2026

1 / 28

Outline

Navigation icons

Lorenz Vaitl et al. (TU Berlin)

Path-Gradient Estimators for CNFs...

May 24, 2026

2 / 28

Generative Modeling & VI

- Generative models power vision, language, physics, chemistry.
- Variational Inference scales Bayesian learning to high dimensions.
- Normalizing Flows enable flexible, tractable density modeling.
- Continuous flows (Neural ODEs) encode symmetries vital in science.

Navigation icons

Lorenz Vaitl et al. (TU Berlin)

Path-Gradient Estimators for CNFs...

May 24, 2026

3 / 28

Why CNFs?

- CNFs use ODE dynamics with free-form Jacobians.
- Exact likelihood via instantaneous change of variables.
- Natural support for equivariance and conservation laws.
- Widely used in image modeling and lattice field theory.

Navigation icons

Lorenz Vaitl et al. (TU Berlin)

Path-Gradient Estimators for CNFs...

May 24, 2026

4 / 28

- Standard total gradients mix path and score terms.
- Score term remains noisy even when q is close to p .
- Variance slows convergence and harms stability.
- CNFs add complexity: gradients flow through ODE paths.

- Compute the gradient of log-density with respect to the final state using a dedicated forward ODE; combine with an adjoint vector–Jacobian product to link to parameters.
- Obtain unbiased, lower-variance path-gradients with constant memory and modest runtime overhead.

Connecting to training: this estimator directly optimizes the chosen objective (e.g., ELBO or reverse KL) by propagating informative, low-variance gradients along learned ODE trajectories.

- Path-gradients reduce variance for simple families.
- Prior CNF path-gradient cloning is memory-heavy.
- Runtime overhead and instability limit adoption.
- Need a CNF-specific, efficient path estimator.

- Continuous-time paths complicate gradient flow.
- We target the state-derivative of log-density.
- Then link to parameters via an adjoint VJP.

Connecting to training: clarifying what is differentiated (state vs. parameters) explains how we control variance in the loss gradient through the ODE path.

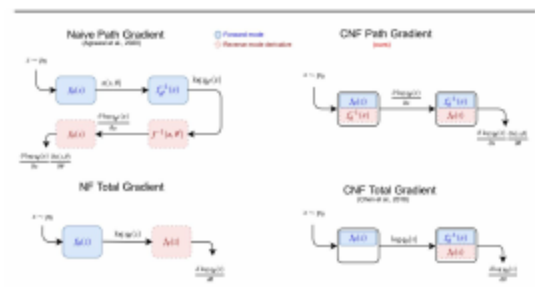


Figure: Schematic comparison of gradients in discrete and continuous flows.

- State evolves under an ODE defined by f_θ over t in $[0, T]$.
- Instantaneous change of variables for log-density:

$$\frac{d}{dt} \ln q = -\text{tr} \left(\frac{\partial f_\theta}{\partial x} \right).$$

- Use Hutchinson's estimator for trace terms; likelihood is base plus divergence integral.

- Train by ELBO or reverse KL.
- Total gradient decomposes into path and score terms.
- Path term captures implicit dependence through dynamics.
- We eliminate the noisy score term in CNFs.

- Derive a forward ODE for the state-gradient of log-density; initialize from the base and integrate alongside the CNF.
- Achieves constant memory with cost similar to a forward pass.

Connecting to training: this provides the sensitivity of the objective to changes in the terminal state, which we then map to parameters.

- Use the adjoint method to form the vector-Jacobian product with parameters without storing intermediate states.
- Backward integration with state recomputation yields the full path-gradient efficiently.

Connecting to training: the adjoint links state sensitivities to parameter updates that directly reduce the training loss.

- About four forward-pass equivalents vs. about three for the standard estimator.
- About half the memory of cloning-based CNF path methods (Agrawal et al., 2020).
- No model cloning; constant memory; overhead decreases with dimensionality.

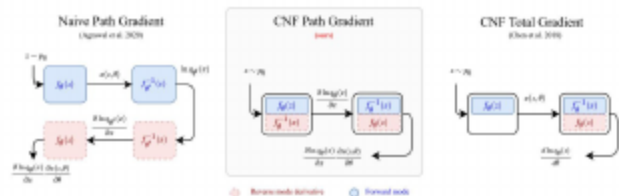


Figure: Comparison of costs: prior cloning-based method vs. ours vs. standard total gradient.

- Hutchinson trace for divergence terms.
- Works with adaptive and fixed-step solvers.
- Large-batch compatible; mixed precision ready.
- Equivariance preserved through dynamics.

- Domains: 2D ϕ^4 lattice field theory; VAE with FFJORD.
- Metrics: ESS and reverse KL; ELBO and NLL.
- Hardware: A100; measure per-iteration runtime and wall-clock convergence.
- Baselines: reparameterized (total) gradient (Kingma & Welling, 2014; Rezende et al., 2014); cloning-based CNF path-gradient (Agrawal et al., 2020).

Baselines cited: Kingma & Welling (2014); Rezende et al. (2014); Grathwohl et al. (2019); Agrawal et al. (2020).

- Path-gradients converge faster across lattice sizes.
- Higher final ESS, with benefits growing in higher dimensions.
- Improved sampling quality and stability.

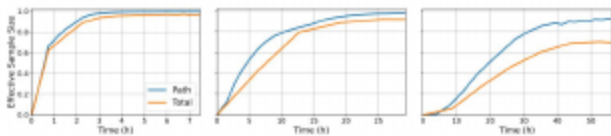


Figure: ESS during training across lattice sizes using different estimators.

- Path-gradient achieves higher ESS at all sizes.
- Fewer epochs yet better sampling efficiency.

Table: Effective Sample Size over 500k samples. Larger is better.

LATTICE SIZE	PATH	TOTAL
12×12	99.66 ± 0.07	98.01 ± 0.44
20×20	97.65 ± 0.14	91.56 ± 1.13
32×32	91.81 ± 1.32	69.53 ± 5.59

- Lower reverse KL at all lattice sizes.
- Largest gap at 32×32, indicating stronger gains in higher dimensions.
- Indicates a closer match to the target distribution.

- Path-gradients show consistently smaller norms.
- Stabler optimization near the optimum.
- Supports the variance reduction hypothesis.

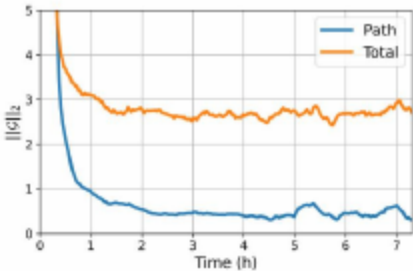


Figure: Gradient norm estimates on a 12-by-12 phi4 lattice (smoothed over runs and steps).

- Overhead decreases with lattice size.
- Faster than prior CNF path approach (Agrawal et al., 2020).
- Wall-clock convergence still improves.

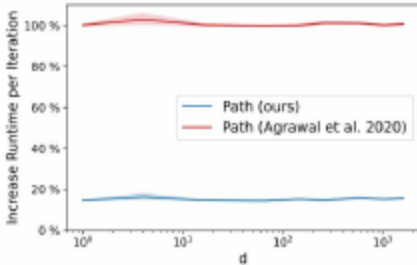


Figure: Per-iteration runtime increase: ours vs. cloning-based prior vs. standard.

Time per Iteration (Table)

- +14% (12×12), +12% (20×20), +5% (32×32).
- Overhead shrinks with dimension.
- Memory savings enable larger batches (cf. Agrawal et al., 2020).

VAE ELBO Results

- Uniform ELBO gains on MNIST and Omniglot.
- Caltech Silhouettes and Frey Faces also improve.
- Demonstrates generality beyond physics (FFJORD baseline per Grathwohl et al., 2019).

Table: Negative ELBO on test data for VAE models using FFJORD flows; lower is better. In nats for all datasets except Frey Faces (bits per dimension). Means and standard deviations over three runs. Baseline results from Grathwohl et al. (2019).

DATASET	PATH	TOTAL
MNIST	82.09 ± 0.04	82.82 ± 0.01
OMNIGLOT	96.61 ± 0.17	98.33 ± 0.09
CALTECH SILHOUETTES	101.93 ± 0.63	104.03 ± 0.43
FREY FACES	4.35 ± 0.00	4.39 ± 0.01

VAE NLL Results

- Lower NLL across datasets evaluated.
- No increase in function evaluations observed.
- Improvements align with ELBO gains, indicating better-calibrated likelihoods.

Solver NFEs

- Adaptive-solver function evaluations remain comparable.
- Estimator does not destabilize integration.
- Supports plug-and-play replacement claims.

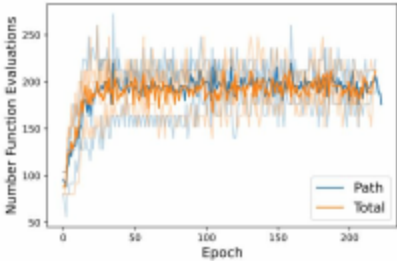


Figure: Number of function evaluations for two estimators under the same hyperparameters.

- Removes the noisy score term near the optimum.
- Stabler updates accelerate late-stage training.
- Benefits grow with dimensionality and tighter solver tolerances.
- Constant memory enables larger batches.

- Greatest gains when q can approach p .
- Mild extra compute vs. total gradients.
- Relies on adjoint recomputation costs.
- Theory beyond near-optimal regimes remains open.

- We thank collaborators and supporting institutions.
- Gratitude to open-source ODE and flow libraries.
- Thanks to the community for insightful feedback.

- When do path-gradients reduce variance early in training, not just near optimality?
- How do they interact with IWAE and alpha-divergences, and with alternative CNF architectures?
- Can solver-aware or mixed-precision designs further cut runtime without bias?
- What theory explains dimension-dependent overhead reductions?